Cobertura y proceso de testing — VESPER, entrega 2

Resultado observado

Ejecución del **5 de noviembre de 2026**, mediante `npm test`: **56 pruebas ejecutadas, 56 aprobadas, 0 fallidas**. Se ejecutaron con Karma 6.4.4 y Jasmine en Chromium Headless 155, sobre Linux y Node.js 24.22.1. Este resultado corresponde al código comprobado en esta entrega; cambios posteriores requieren una nueva ejecución.

Archivo de pruebas	Casos	Alcance
`tests/components.spec.jsx`	32	Props, renderizado, estado, DOM, filtros, formularios, carrito, checkout, resultados de pago y acceso
`tests/admin.spec.jsx`	7	Guardas de administración, navegación, creación de productos y CRUD de categorías
`tests/store.spec.js`	17	Persistencia, migración, autenticación, CRUD, stock, validación y atomicidad de compras
Total	**56**	**Todos aprobados**

Resumen de la salida real:

```
TOTAL: 56 SUCCESS
Statements   : 69.09% ( 494/715 )
Branches     : 68.77% ( 392/570 )
Functions    : 66.42% ( 182/274 )
Lines        : 70.6% ( 442/626 )
```

Cobertura instrumental

La instrumentación Babel/Istanbul incluye **todos los módulos JavaScript/JSX de `src`**, aunque no tengan pruebas directas. `tests/index.js` los incorpora explícitamente al denominador. Se excluye únicamente `src/main.jsx`, que monta la raíz de React, además de CSS, imágenes, dependencias y los propios tests. Cargar un módulo de datos ejecuta sus declaraciones; una cobertura alta en esos archivos no implica que sus contenidos hayan sido revisados semánticamente.

Métrica	Elementos cubiertos / total	Cobertura
Sentencias	494 / 715	69.09 %
Ramas	392 / 570	68.77 %
Funciones	182 / 274	66.42 %
Líneas	442 / 626	70.6 %

Los porcentajes de sentencias, ramas, funciones y líneas miden cosas distintas. Una línea JSX puede contener varias decisiones y callbacks, por lo que no debe equipararse el porcentaje de líneas con una validación completa de la aplicación.

Módulo	Sentencias	Ramas	Líneas
`src/App.jsx`	0 %	No aplica	0 %
`src/components/Feedback.jsx`	100 %	100 %	100 %
`src/components/Field.jsx`	100 %	100 %	100 %
`src/components/Layout.jsx`	7.69 %	18.18 %	7.69 %
`src/components/ProductCard.jsx`	100 %	100 %	100 %
`src/components/ProductList.jsx`	100 %	100 %	100 %
`src/context/ShopContext.jsx`	85.71 %	33.33 %	100 %
`src/data/articles.js`	100 %	No aplica	100 %
`src/data/seed.js`	100 %	100 %	100 %
`src/pages/AdminPages.jsx`	49.73 %	41.37 %	54.08 %
`src/pages/AuthPages.jsx`	44.18 %	58.49 %	46.15 %
`src/pages/CartPages.jsx`	89.65 %	73.8 %	90.56 %
`src/pages/CatalogPage.jsx`	69.56 %	89.28 %	69.56 %
`src/pages/ContactPage.jsx`	100 %	66.66 %	100 %
`src/pages/InfoPages.jsx`	0 %	0 %	0 %
`src/pages/ProductPage.jsx`	0 %	0 %	0 %
`src/services/store.js`	87.13 %	82.65 %	87.08 %

«No aplica» indica que el módulo no contiene decisiones instrumentables para esa métrica.

Los informes generados son `coverage/index.html`, `coverage/lcov.info` y `coverage/coverage-summary.json`. Estos archivos se regeneran al ejecutar la suite y no forman parte del código versionado.

Relación con los requisitos y casos demostrables

Requisito / comportamiento	Evidencia automatizada	Resultado esperado
Renderizado de listas	`renderiza todos los productos de una lista`	Todos los productos entregados aparecen en el DOM
Props de componentes	`renderiza los datos del producto recibidos por props`	Nombre, imagen y precio reflejan el producto
Eventos de usuario	`ejecuta una sola vez el callback al agregar el producto`	Un clic llama a `onAdd` con el identificador correcto

Requisito / comportamiento	Evidencia automatizada	Resultado esperado
Estado controlado	`actualiza el DOM cuando cambia el estado de un formulario controlado`	El campo y el texto visible reflejan la entrada
Renderizado condicional	`oculta el mensaje de error cuando no hay error` y `muestra un error condicional con rol de alerta`	Solo se presenta un error si existe
Disponibilidad	`bloquea la compra de un producto sin stock`	Botón deshabilitado y callback sin ejecutar
Filtros y ofertas	Casos de búsqueda, categoría, ofertas y ausencia de coincidencias	La lista responde a los criterios elegidos
Formularios React	Casos de contacto correcto y validación fallida	Se envían datos controlados, se presenta el resultado y se conserva el texto ante error
Carrito	Casos de subtotal, cantidad, eliminación y carrito vacío	Total coherente y llamadas al servicio con el producto seleccionado
Checkout	Autocompletado, retiro, aprobación y rechazo	Datos de perfil precargados, envío coherente y navegación al resultado correspondiente
Compra exitosa	`genera pedido con descuento, envío e instantánea independiente del catálogo`	Descuenta stock, vacía carrito y conserva el pedido aunque cambie el catálogo
Compra rechazada	`conserva carrito, stock y pedidos al rechazar un pago`	No modifica existencias, carrito ni pedidos
Persistencia	Migración, recarga y almacenamiento corrupto	Recupera datos válidos y conserva cambios previos
CRUD administrativo	Pruebas de producto, categorías y usuarios	Alta, lectura, modificación y eliminación coherentes
Control de acceso local	Casos de visitante, cliente y administrador	El visitante va al login y el cliente no ve los mantenedores
Acceso de usuario	Credenciales inválidas, contraseñas distintas y login correcto	Errores visibles y navegación al perfil tras ingresar
Manejo de fallos	`conserva el estado si la persistencia falla durante una compra`	Un error de almacenamiento no deja una compra parcialmente aplicada

Esta matriz proporciona más de los diez casos solicitados por la rúbrica y contempla entradas válidas, inválidas y estados vacíos.

Herramientas y configuración

- **Jasmine** define suites (`describe`), casos (`it`), expectativas y espías (`jasmine.createSpy` / `spyOn`).
- **Karma** inicia el servidor de pruebas y abre un navegador real. `karma-chrome-launcher` usa Chromium/Chrome Headless.
- **Webpack y Babel** transforman JSX y módulos para el navegador. `babel-plugin-istanbul` instrumenta únicamente el código de aplicación.
- **React `createRoot` y `act`** montan los componentes, aplican eventos y esperan las actualizaciones antes de comprobar el DOM.
- **MemoryRouter** reproduce navegación sin depender de un servidor de rutas.

- **ShopProvider** recibe un store aislado o una instantánea estable, lo que permite probar las vistas sin cuentas reales.

`tests/helpers.jsx` crea y desmonta una raíz por caso. Los cambios de campos usan el setter nativo y eventos `input`/`change`; los formularios reciben un evento `submit` cancelable. Esto comprueba el comportamiento observable de los componentes.

Mocks y aislamiento

Las pruebas de servicio crean almacenamiento en memoria en cada `beforeEach`, sin modificar los datos de un usuario real. Las de administración combinan ese store aislado con espías que delegan las mutaciones al mismo store para observar el renderizado tras CRUD.

Los casos de contacto, acceso y navegación de pago simulan respuestas y errores del servicio con `spyOn`. Comprueban la reacción de la interfaz; las reglas del pago y la persistencia se comprueban por separado con el servicio real e instancias aisladas. Así se distingue un fallo de validación del servicio de un fallo de presentación o navegación.

`afterEach` desmonta React y retira el contenedor del DOM. Jasmine restaura los espías entre casos. La semilla de productos se mantiene independiente de los datos de cada prueba.

Reproducción y regresión

```
npm ci
npm test
npm run build
```

Para mantener el navegador escuchando mientras se trabaja:

```
npm run test:watch
```

Se requiere Node.js 22.12 o superior y Chrome/Chromium instalado. El archivo `karma.conf.cjs` detecta `/usr/bin/chromium`; en otra ubicación se indica explícitamente:

```
CHROME_BIN=/ruta/al/chromium npm test
```

En Windows PowerShell puede establecerse `$env:CHROME_BIN` antes del comando. En este entorno aislado se habilitó acceso de red para que Karma pudiera escuchar en el puerto local; no se enviaron datos del proyecto a un servicio de testing.

Proceso aplicado: identificar comportamiento y criterio de aceptación; preparar datos aislados; ejecutar la acción; comprobar resultado; corregir el defecto si aparece; volver a ejecutar los casos afectados y finalmente toda la suite. Antes de entregar, repetir la suite y la compilación sobre la versión final. Si falla un test, revisar su nombre, la expectativa y el stack del navegador. Para localizar rutas sin ejecutar, abrir el informe HTML de cobertura.

Límites y verificación manual complementaria

La suite **no acredita cobertura del 100 %**. El layout, las páginas informativas y el detalle de producto tienen ejecución automatizada limitada o nula; el panel administrativo no tiene todos sus formularios y reportes cubiertos. Las rutas principales se prueban parcialmente. El CSS responsivo, contraste, navegación por teclado, impresión y lectura de pantalla requieren revisión visual y manual adicional.

El servicio es una base de datos simulada en JavaScript. Estas pruebas no verifican una API, correo, pasarela bancaria ni seguridad de servidor. Las guardas locales y los roles son parte de la demostración académica; no constituyen autorización segura para producción.

Recorrido manual complementario recomendado: tienda sin sesión; búsqueda y categorías; cuenta de cliente y perfil; carrito y checkout con despacho y retiro; pago rechazado y reintento; compra exitosa y comprobante; administración de productos, categorías y usuarios; pedido y reportes. Repetir en móvil, tableta y escritorio, recargando después de cambiar datos para comprobar persistencia. Los resultados manuales o E2E deben registrarse por separado y no incrementan estos porcentajes de cobertura unitaria.

Comprobación adicional en navegador real

El 5 de noviembre de 2026 se ejecutó un recorrido automatizado con Playwright y Chromium contra la compilación de producción (`npm run preview`, puerto 4173): **48 comprobaciones aprobadas, sin errores de consola o de página**. El registro está en `docs/evidencias/verificacion-navegador.json`. Este recorrido se ejecutó con las herramientas del entorno de desarrollo y no sustituye ni se suma a las 56 pruebas de Jasmine/Karma.

Se comprobaron catálogo público y filtros, detalle, carrito persistente y cantidades, validación de checkout, pago rechazado sin alterar carrito/stock/pedidos, compra aprobada y comprobante, perfil y autocompletado, historial, roles, CRUD de administración, reportes, registro y contacto.

Se midió ausencia de desbordamiento horizontal de la página a **390, 768 y 1440 px**, incluyendo carrito con productos, checkout y administración. Se corrigió el posicionamiento de los textos accesibles de las tablas administrativas para mantenerlos dentro del contenedor desplazable. Las tablas pueden tener desplazamiento interno intencional en móvil.

Capturas guardadas:

- `docs/evidencias/inicio-escritorio.png`
- `docs/evidencias/inicio-movil.png`
- `docs/evidencias/checkout-escritorio.png`
- `docs/evidencias/administracion-escritorio.png`

Estas comprobaciones no son una auditoría completa de accesibilidad ni incluyen todas las combinaciones de navegadores y tamaños. Sus resultados no modifican los porcentajes instrumentados de cobertura unitaria.